

DroidSpeak: KV Cache Sharing Across Fine-tuned Model Variants

Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du,

Hanchen Li, Yihua Cheng, and **Junchen Jiang**, University of Chicago;

Shan Lu, Madan Musuvathi, and Esha Choukse, Microsoft

汇报人：刘翊



作者

研究方向：

为大模型构建高效的推理系统

近期工作：

- CacheBlend: Fast Large Language Model Serving with Cached Knowledge Fusion EuroSys'25(Best Paper Award)：提出了一种在RAG场景下突破前缀限制的KV Cache复用与融合方案
- CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving SIGCOMM'24：提出了一种面向大模型服务的KV Cache压缩与流式传输方案



Yuhan Liu

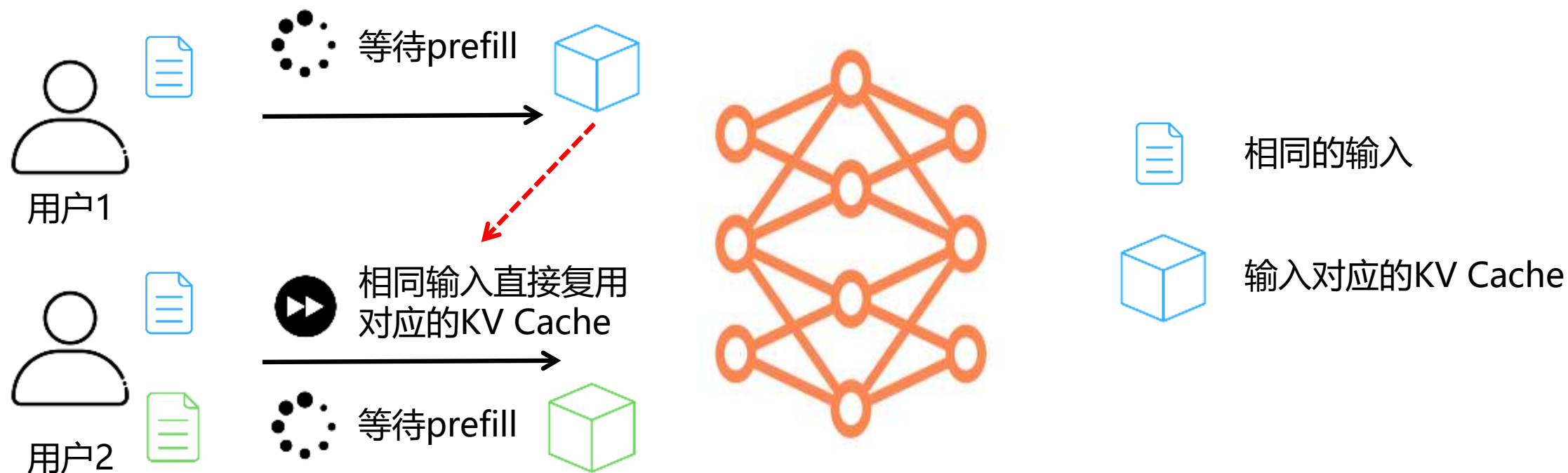
<https://yuhanliu11.github.io/>

概要

- 研究背景
- 动机
- 系统设计
- 实验
- 思考

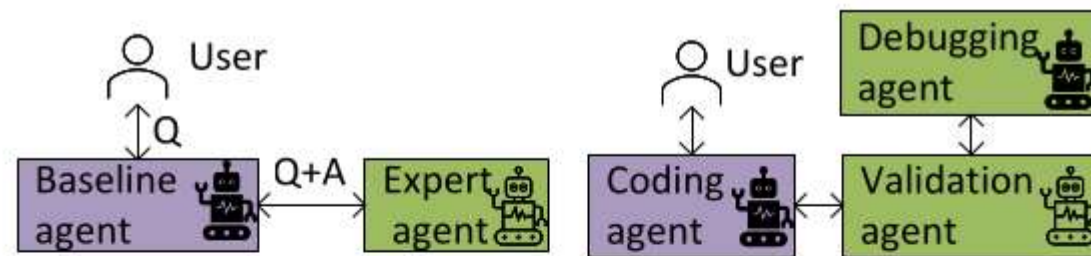
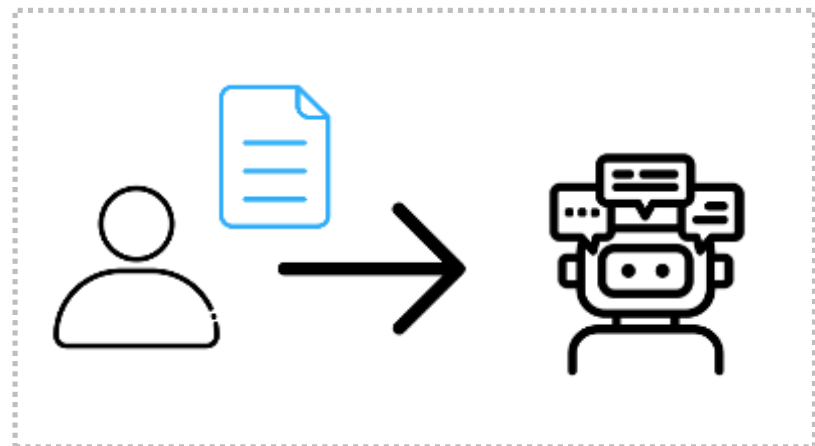
研究背景：KV Cache复用

大模型推理的计算开销逐渐增长，现有的很多工作通过相同模型处理相同输入前缀时复用对应的KV Cache的方式节省计算开销

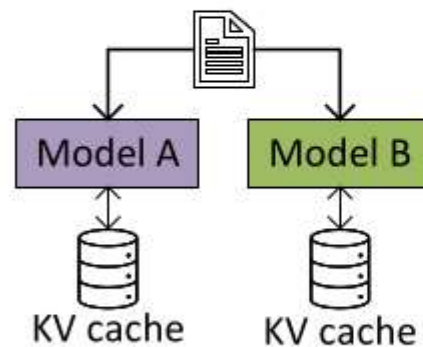


研究背景：多个大模型协同

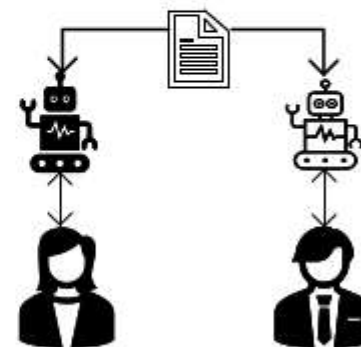
而目前大模型服务逐渐从单个模型处理变为多个模型协同处理，这些协作的模型之间也有很多相同的输入需要重复prefill，但这些协作的模型不一定是相同的



A) Workflows with multiple LLM agents



B) Multiple models working on same content, storing their own KV cache

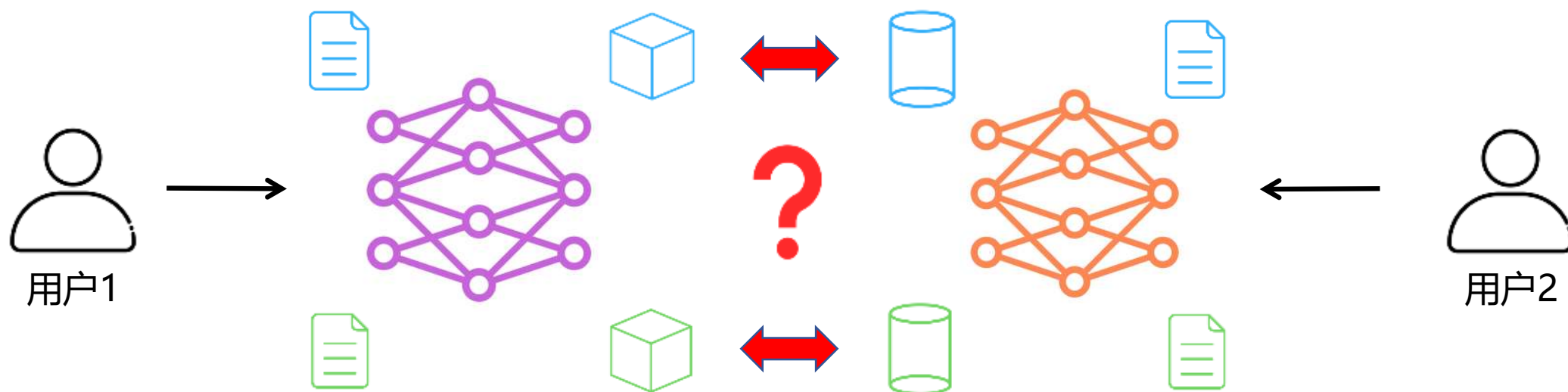


C) Personalized agents accessing the same content

研究背景：不同模型之间的KV Cache复用



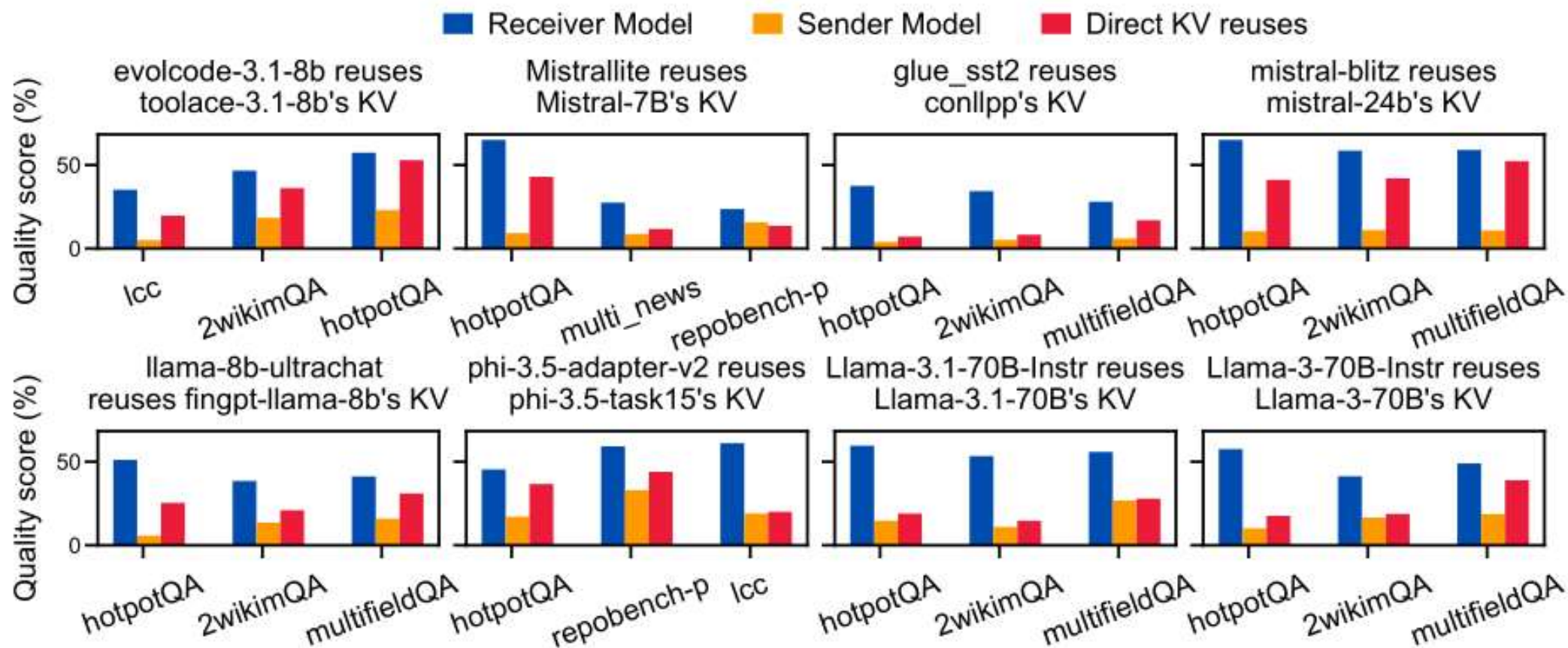
能否在不同的模型之间复用相同输入的KV Cache？



本文的不同模型只针对相同基座的不同微调版本的模型

动机：直接复用完整KV Cache

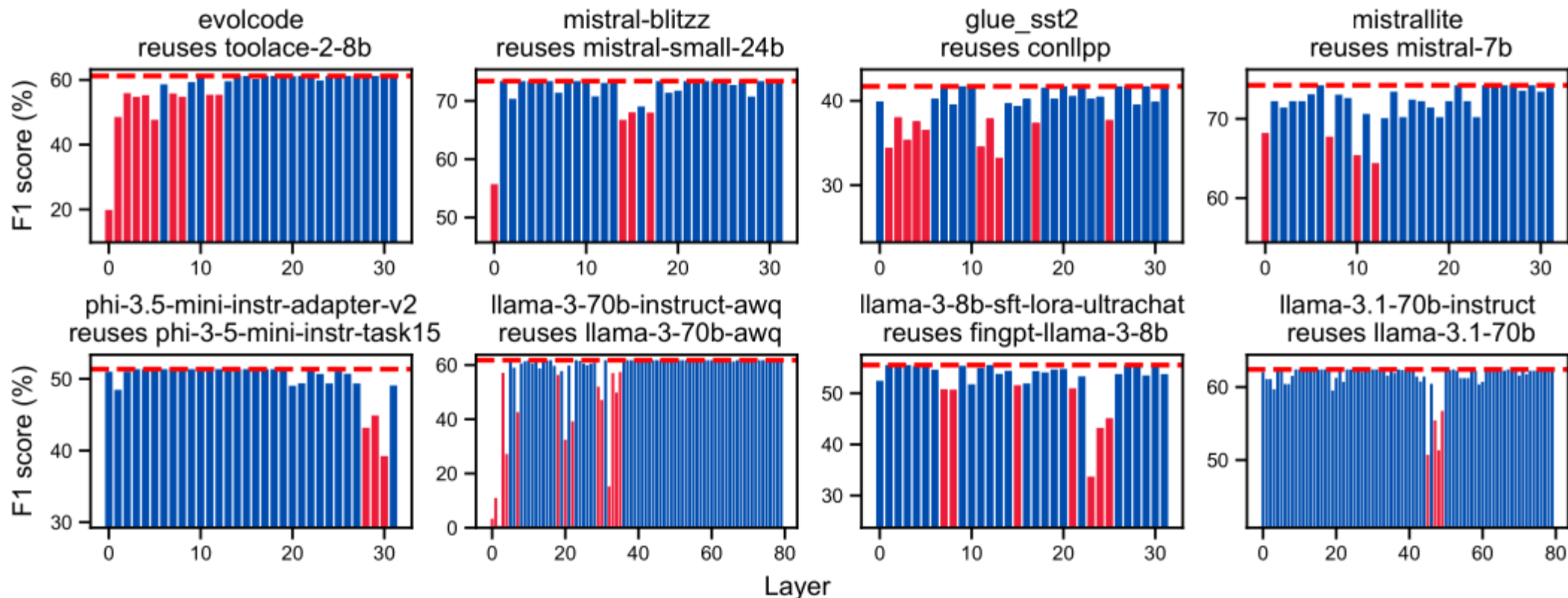
直接复用完整的KV Cache在多组模型和数据集上都有明显的质量下降



动机：复用部分层的KV Cache



那只复用部分层的KV Cache呢？

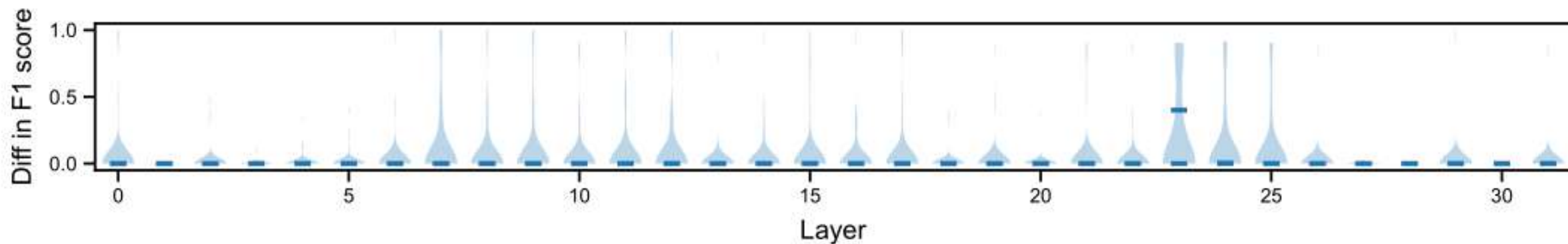


只有部分层比较关键！90%的层的KV Cache直接复用质量损失很小
因此可以只重计算这些关键层的KV、大部分层直接复用已有的KV

动机：层敏感性与输入无关



那这些关键层会和输入相关吗？

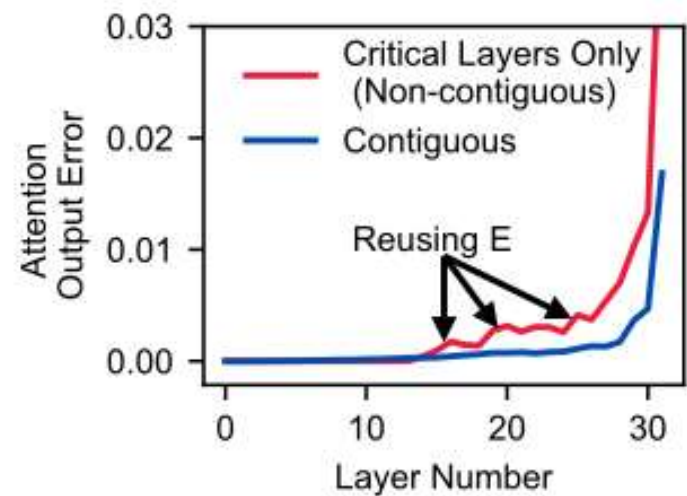
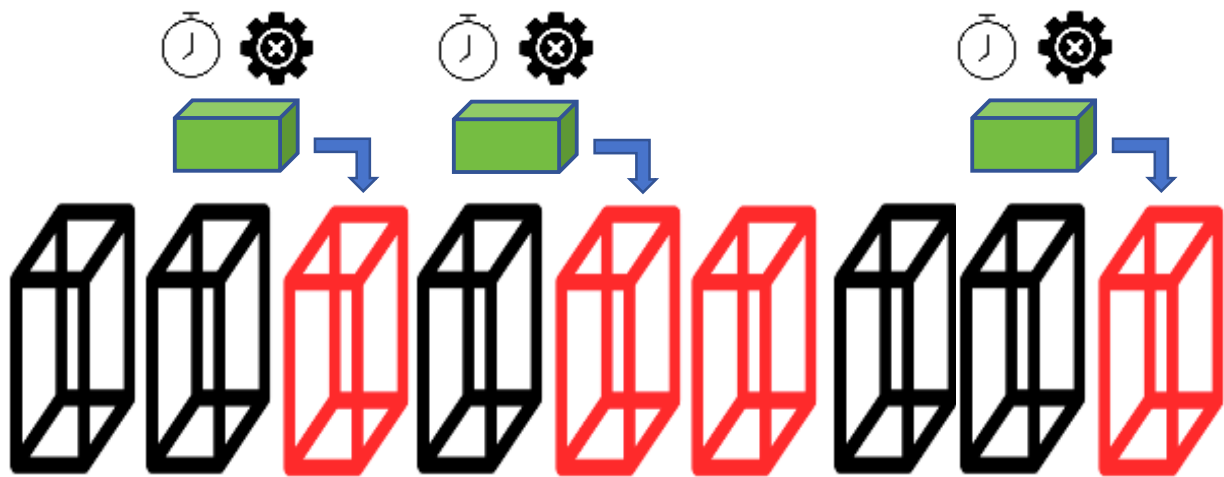


同一个模型对，不同输入上表现出的关键层是相似的！

因此可以离线确定某些模型对的关键层，不需要为每个请求重新分析

动机：重计算部分连续的关键层

直接针对所有关键层都重计算不是最佳方案，因为关键层会分散在多个不连续的位置，每次重计算都需要对应的发送模型该层输入的Embedding，多次输入Embedding会同时增加传输开销和输出误差。



🤔 那怎么决定重计算哪些关键层？

动机：实验发现到本文设计

实验发现

完整复用KV Cache会显著降低生成质量



不同层的敏感性差异明显，
但同一模型对规律相对稳定



针对分散的关键层都进行重计算会降低质量和效率



本文设计

仅复用部分层的KV Cache，对关键层重计算

针对每个模型对进行离线Profile
将配置用于在线请求

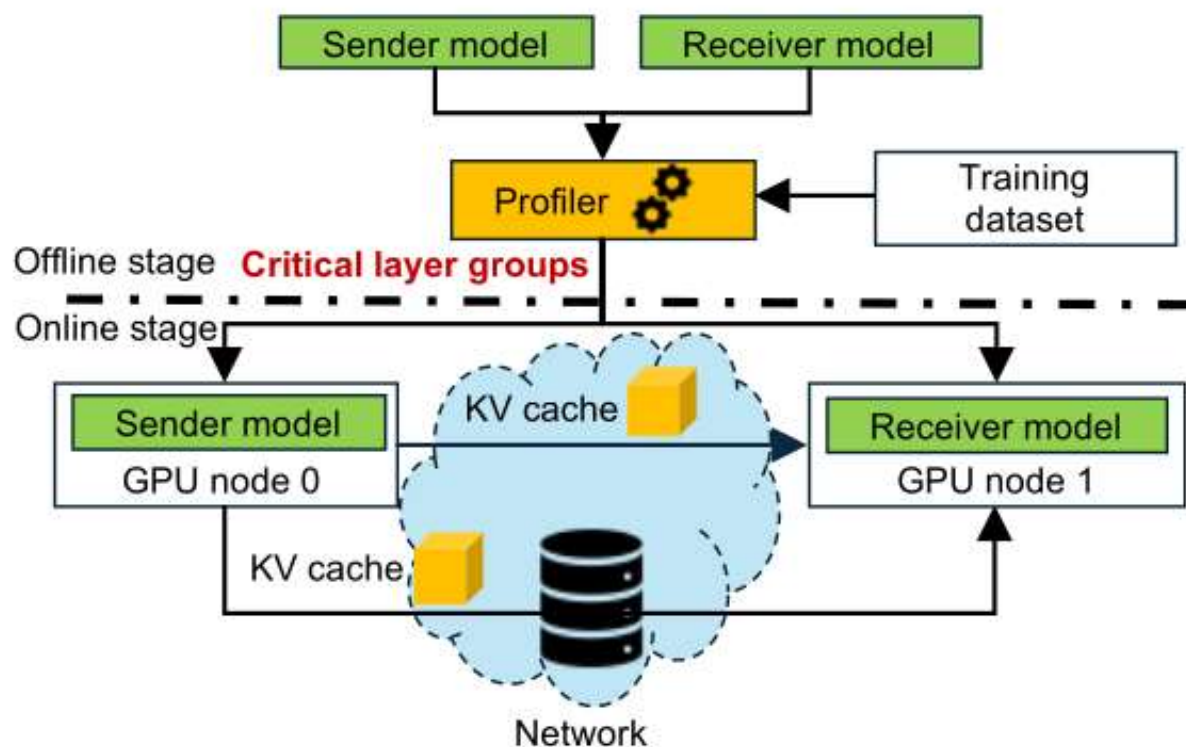
选择一个连续的重计算区间，只加载一次过渡层的E Cache

设计：系统框架

DroidSpeak包含两个阶段：

离线阶段：为每个模型对确定需要重计算的关键层

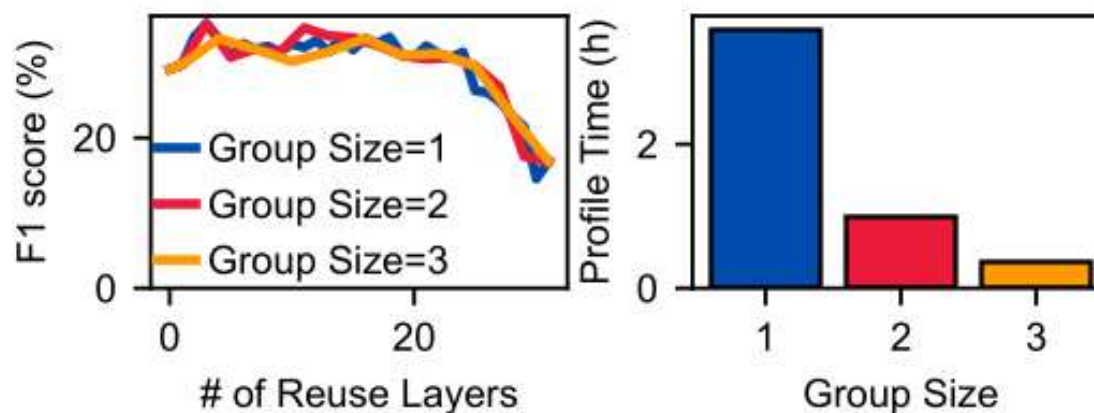
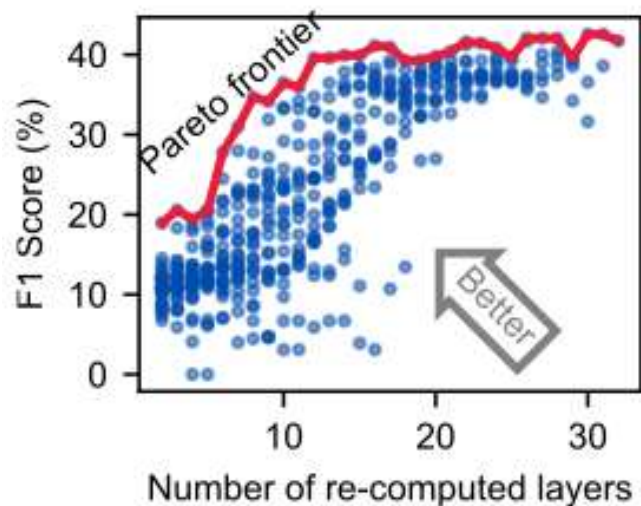
在线阶段：加载可复用的KV Cache并重计算对应的关键层



设计：离线阶段

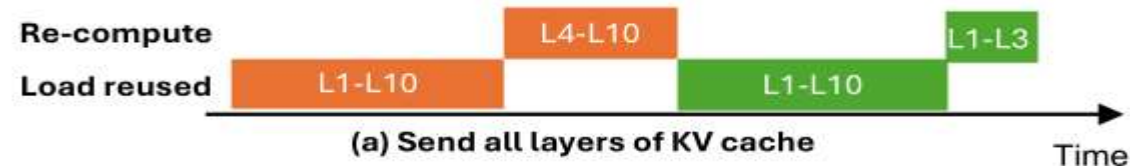
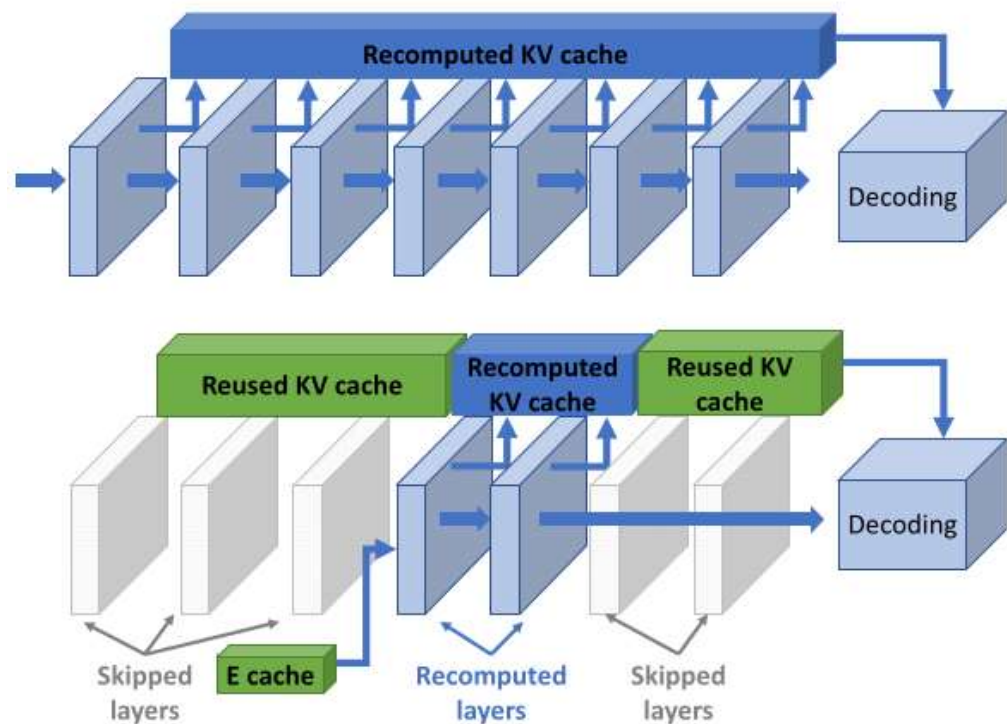
离线阶段枚举重计算的区间，记录每种重计算层数对应的最高质量的区间配置

为了降低枚举的时间开销，本文将相邻层拼成一组后按组来枚举，在保持配置质量的前提下显著缩短分析时间



设计：在线阶段

在线阶段根据延迟目标选择离线生成的重计算配置，接收模型从过渡层E Cache开始重计算连续层区间，加载并复用其他层的KV Cache进一步通过将复用KV Cache和重计算并行处理的方式降低总处理时间



实验设置

实验环境：

本文基于vLLM实现，系统包含3000行Python代码

使用两台设备各配8张80GB的A100，设备间传输带宽25GB/s

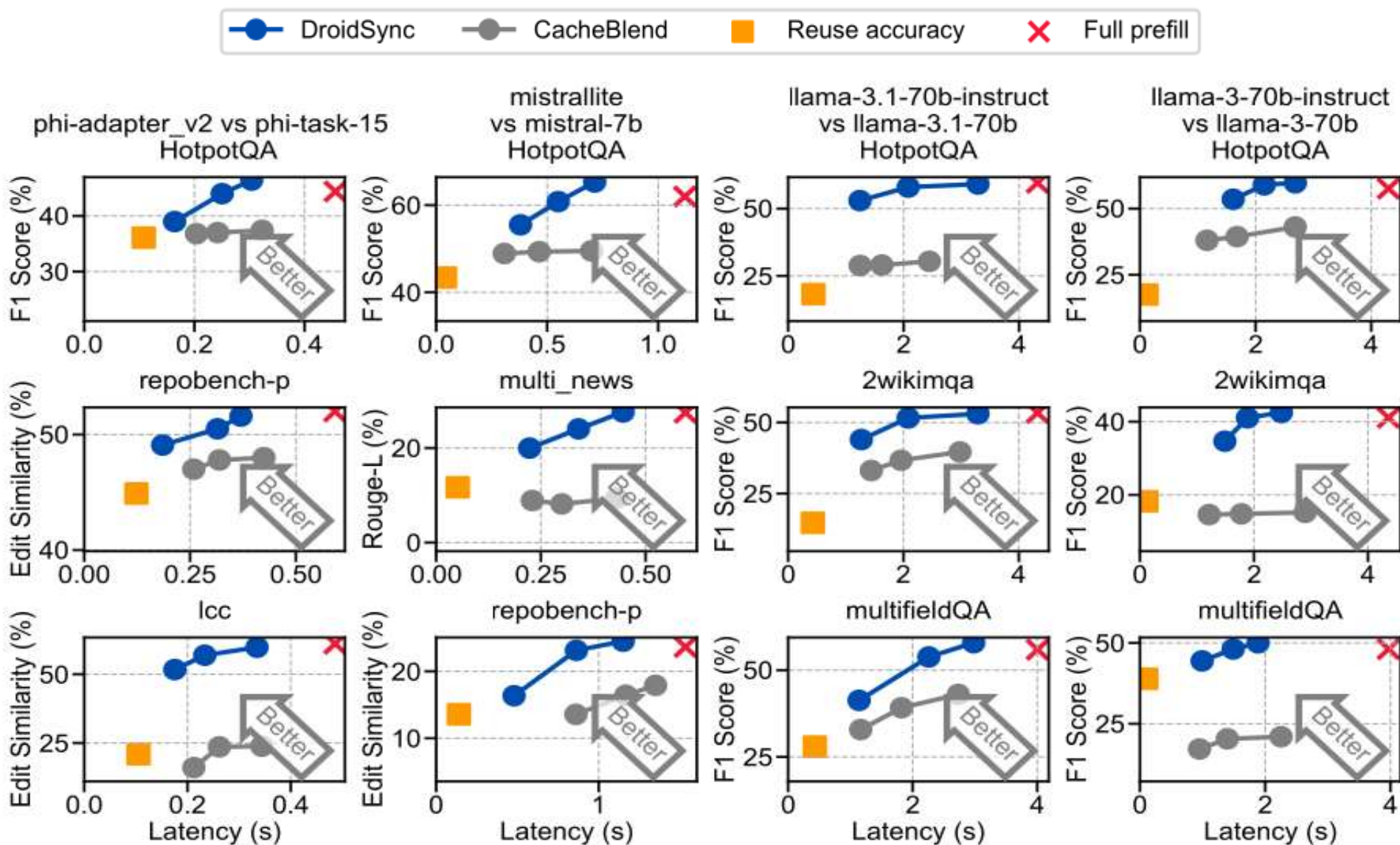
数据集	任务领域
HotpotQA	开放领域问答
2WikiMQA	维基百科的问答
MultiFieldQA_en	多领域长文本问答
MultiNews	多文档新闻摘要
LCC	长上下文代码补全
RepoBench-P	仓库级代码补全

Baseline：

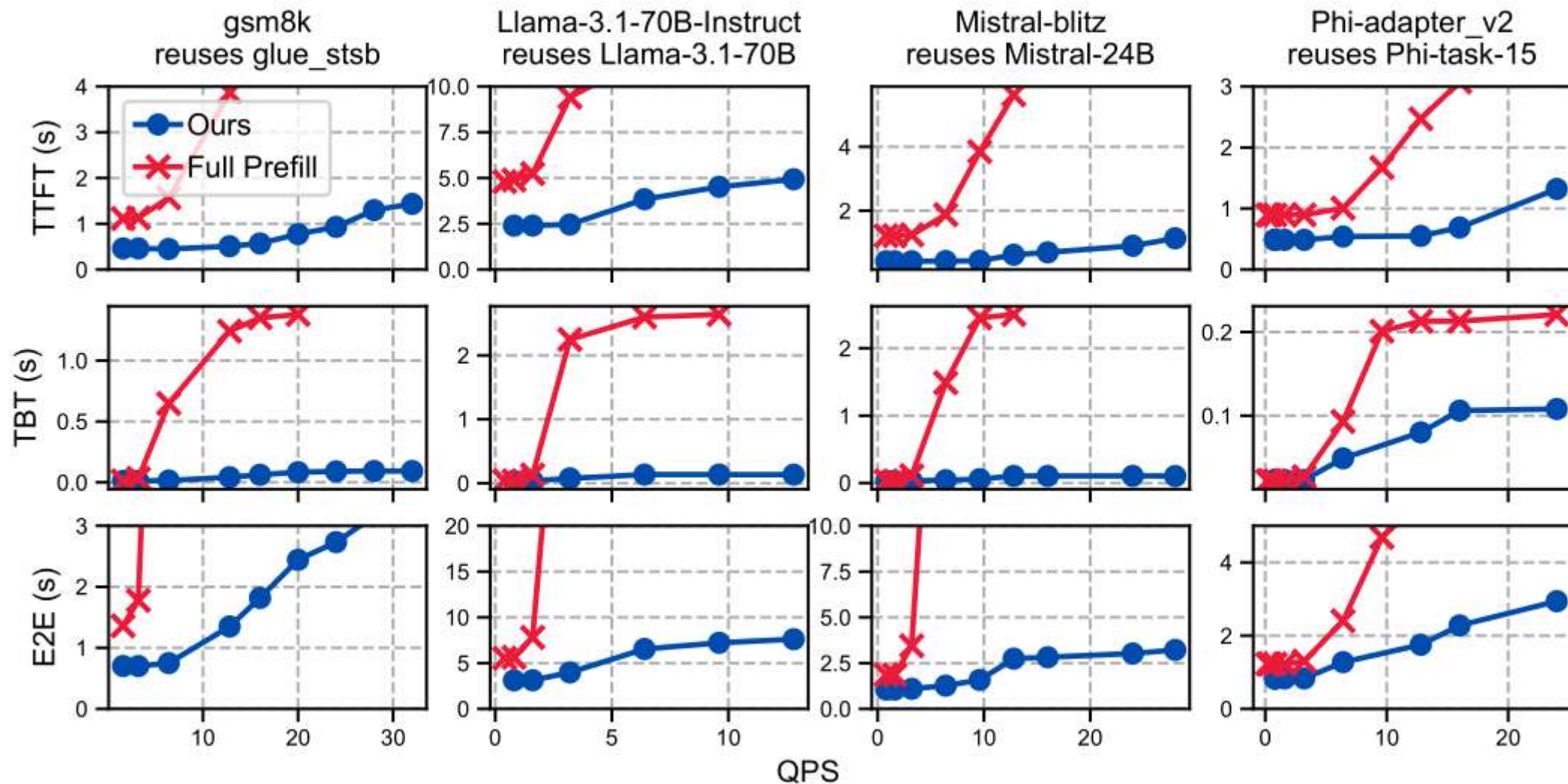
- 本文方法：关键层重算、其余复用
- 不复用KV Cache，完整执行prefill
- 直接复用完整的KV Cache
- 改造CacheBlend为跨模型版本

Receiver Model	Sender Model
evolcode-3.1-8b	toolace-3.1-8b
glue_sst2	conllpp
mistral-blitz	mistral-24b
phi3.5-mini-instr-adapter-v2	phi-3.5-mini-instr-task15
llama-3-8b-sft-lora-ultrachat	finppt-llama-3-8b
llama-3-70b-instruct	llama-3-70b
mistral-lite	mistral-7b
llama-3.1-70b-instruct	llama-3.1-70b

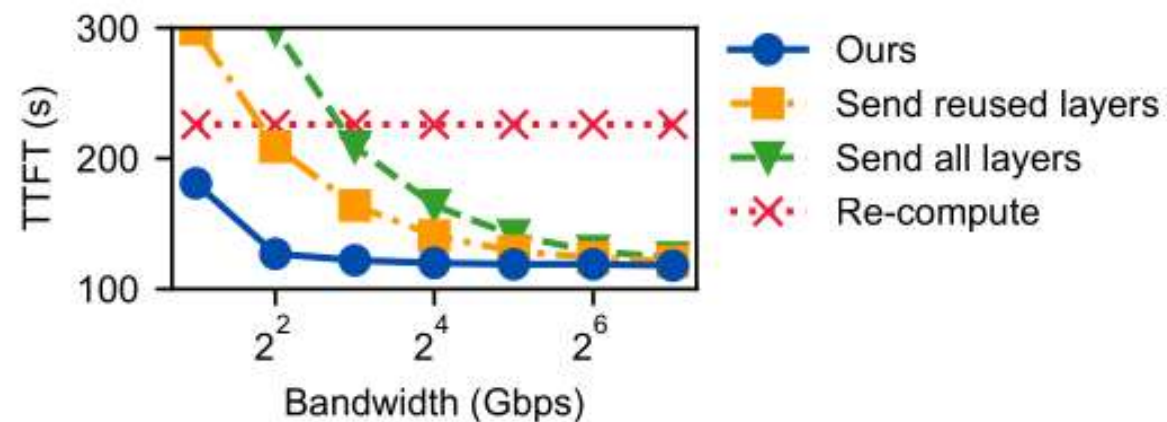
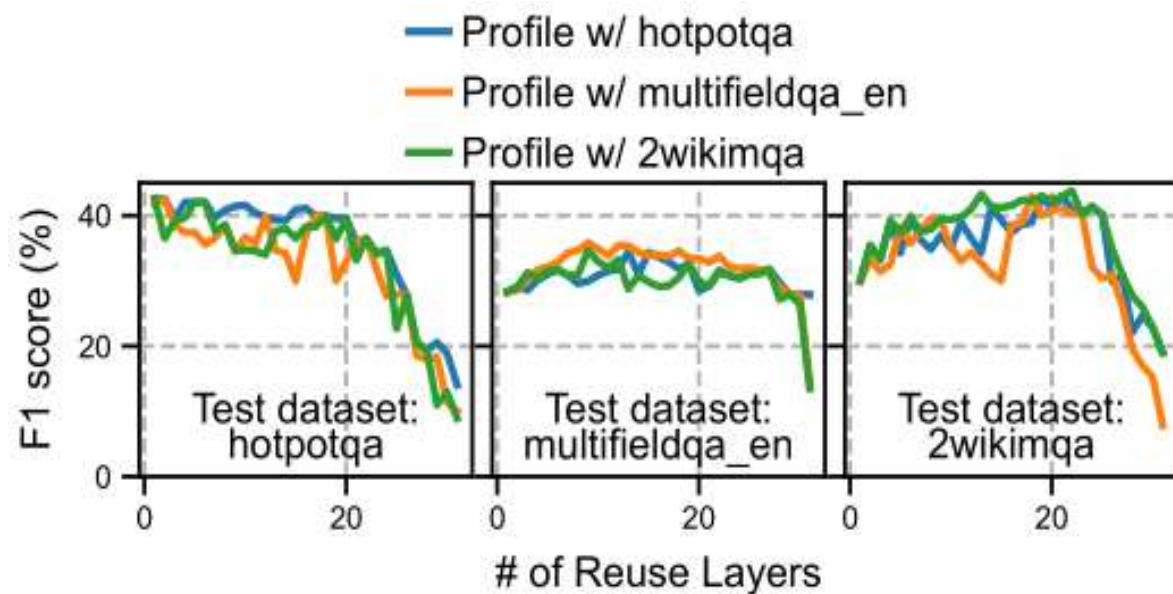
实验结果：Prefill延迟与生成质量



实验结果：在线吞吐与请求延迟



实验结果：跨数据集的鲁棒性和带宽收益



思考

能否进一步提高：

- 1.KV Cache在传输时可以进一步加入压缩策略的维度，也有部分相关工作是联合考虑质量与效率寻找最优KV Cache压缩策略，可以结合起来考虑。
- 2.本文考虑的KV Cache复用的细粒度是逐层对应的，如果考虑的更细一点，可能该关键层的不同注意力头的影响程度是不同的。

能否泛化：

- 1.将KV Cache复用进一步扩展到相同家族不同规模的模型之间，需要处理层数、注意力头数和隐藏维度上的问题。
- 2.将复用相同上下文的KV Cache扩展为两个协作模型各自处理一半上下文的KV Cache融合。

思考

能否用到我们的场景：

在端侧设备协同场景中，不同设备的算力、内存、带宽和剩余电量差异较大，它们可能需要处理相同的上下文。如果一个设备已经使用某个模型处理了共享上下文，另一个设备运行同一模型家族的专用版本。第二个设备可以接收前一个设备的KV Cache，只重计算关键层，从而减少重复Prefill。

系统需要根据实时状态动态决策：如果接收设备算力较弱但网络条件较好，可以复用更多Cache；如果无线链路带宽较低而本地计算资源充足，可以压缩 Cache 或增加本地重计算。

Thanks